

내부원장 Java 구현 internal-ledger 서비스

영구 금융 원장 · NFT 보유 이력 · SHA-256 감사 로그
Spring Boot 3.2 · JPA · 영구 원장 DB · Circuit Breaker

Java 17

Spring Boot 3.2

영구 원장 DB

JPA / Hibernate

TypeScript가 하는 것 vs Java가 하는 것

두 레이어의 책임 경계 — 임시 상태(PostgreSQL) vs 영구 원장(운영 DB)

TypeScript (Node.js · PostgreSQL)

담당	용도
mint_requests	TX 상태 추적 (in-flight)
processed_events	이벤트 중복 방지
AuditLogService	단일 레코드 SHA-256 체크섬
user_nft_holdings	Reconcile 조회용 (읽기)
임시·운영용 — FINALIZED 이후 Java로 위임	

Java (Spring Boot · 영구 원장 DB)

담당	용도
user_nft_holdings	영구 원장 (authoritative) — 쓰기
audit_log	SHA-256 hash chain — append-only
영구(permanent) · 가상자산이용자보호법 §15 5년 보존 user_nft_holdings 단일 진실 원천 = 운영 DB	

TypeScript → Java 호출 구조

FINALIZED 이후 Node.js가 HTTP로 Java internal-ledger에 두 가지를 위임한다

Node.js (TypeScript)

LedgerService.updateMintRequest()

→ status: FINALIZED

ICoreBankingAdapter (interface)

KyoboCoreBankingAdapter

InternalGatewayClient

POST

/users/{userId}
/nft-holdings

POST

/audit-log



HTTP
내부망

X-Internal
-Secret

Java (Spring Boot)

BlockchainGatewayController

InternalLedgerService

중복 방지 → save()

NftHoldingRepository

AuditLogService

REQUIRES_NEW TX

AuditLogRepository

영구 원장 DB (개발: PostgreSQL / 운영: 교보 내부망 DB)

두 가지 위임: ① NFT 보유 이력 → `user_nft_holdings` INSERT | ② 감사 로그 → `audit_log` append-only INSERT

왜 Java인가

기술 선택이 아니라 규제와 기존 인프라 결정 — DB는 JPA로 추상화 (개발: PostgreSQL / 운영: 교보 내부망 DB)

이유	설명	연결 규제·요건
교보생명 Core Banking 연동	기존 Java 스택과 동일 언어 — 내부망 직접 연결, 팀 간 기술 장벽 없음	내부망 데이터 이동 최소화
금융사 표준 기술 스택	Java EE → Spring Boot 전환이 금융권 표준 경로. 감사·컴플라이언스 팀 친숙	금감원 전자금융감독규정
강력한 트랜잭션 보장	JPA @Transactional + SELECT FOR UPDATE — 금융 ACID 요건 충족	가상자산이용자보호법 §15 5년 보존
@Transactional REQUIRES_NEW	감사 로그는 메인 TX 롤백 시에도 반드시 남아야 함	ISMS-P A.9 접근 제어 로그 보존
JPA Sequence 전략	DB Sequence → 연속 ID 보장. Hash chain prevChecksum 순서 의존	감사 로그 순서 무결성

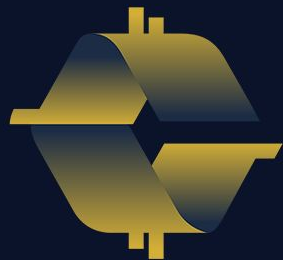
요약: Node.js는 블록체인 이벤트 처리(빠름·경량)에 최적화. Java는 금융 규제 영구 저장(안전·검증)에 최적화. 두 레이어가 역할을 분담한다.

1 API 계약

내부망 엔드포인트 · DTO · 보안 헤더

4개 엔드포인트

X-Internal-Secret



COINCRAFT
ACADEMY

API 계약 — 4개 엔드포인트

BlockchainGatewayController · @RequestMapping("/api/internal")

엔드포인트	메서드	역할	호출 시점
GET /users/{userId}	GET	KYC 상태 + 지갑 주소 조회 — NFT 발행 전 검증	mint 요청 수신 시
POST /users/{userId}/nft-holdings	POST	NFT 취득 이력 영구 원장 DB 영구 기록	on-chain Transfer FINALIZED
POST /audit-log	POST	감사 로그 append-only 기록 (SHA-256 chain)	모든 상태 변경 시
POST /rewards/notify	POST	NFT 발행 완료 → Core Banking 포인트/혜택 지급 알림	FINALIZED 후

보안 — 인증 방식

X-Internal-Secret 헤더 필수

값은 환경변수 **INTERNAL_GATEWAY_SECRET** 주입

하드코딩 절대 금지 · 향후 mTLS 전환 예정

접근 제어 — 네트워크 레벨

이 Controller는 내부망 IP에서만 접근 가능

방화벽 레벨에서 외부 차단

Node.js issuer-service만 호출 허용

POST /nft-holdings — NftHoldingRequest DTO

Java record + Bean Validation — @NotNull · @NotBlank · @Min

```
// Java 17 record - 불변 DTO
public record NftHoldingRequest(
    @NotNull Long    tokenId,
    @NotBlank String contractAddr,
    @NotNull Integer chainId,
    @NotNull
    @Min(1) Long    amount,
    @NotNull Instant acquiredAt,
    @NotBlank String onChainTx
) {}
```

필드	타입	설명
tokenId	Long	ERC-1155 token ID
contractAddr	String	NFT 컨트랙트 주소 (0x...)
chainId	Integer	블록체인 체인 ID
amount	Long	발행 수량 (최소 1)
acquiredAt	Instant	온체인 취득 시각 (UTC)
onChainTx	String	TX 해시 (66자, 0x...)

Node.js 측 미리 타입: GatewayNftHoldingRequest

Java Instant → TS의 ISO-8601 string으로 직렬화

POST /audit-log — AuditLogRequest DTO

Node.js AuditLogService가 Java로 위임 — HTTP POST 직렬화

```
public record AuditLogRequest(  
    @NotBlank String actor,  
    @NotBlank String action,  
    @NotBlank String resourceType,  
    @NotBlank String resourceId,  
    String beforeState, // nullable  
    @NotBlank String afterState  
) {}
```

필드	설명
actor	행위자 (operator ID, "SYSTEM")
action	이벤트 코드 (NFT_ACQUIRED, RECONCILE_MISMATCH_DETECTED 등)
resourceType	대상 엔티티 유형 (USER, NftHolding...)
resourceId	대상 ID (userId, contractAddr:tokenId)
beforeState	변경 전 JSON — 신규 생성 시 null
afterState	변경 후 JSON — checksum 입력값
SHA-256 입력: prevChecksum + eventTime + actor + action + resourceId + afterState beforeState 는 체크섬 대상 아님 — 참조용 스냅샷	

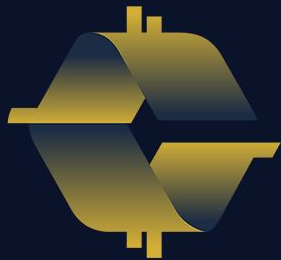
2

Entity & DDL

NftHolding · AuditLogEntry · 설계 결정

@Entity

운영 DB Sequence



COINCRAFT
ACADEMY

NftHolding @Entity

user_nft_holdings 테이블 · 교보 영구 원장 DB 영구 보관

```
@Entity
@Table(name = "user_nft_holdings",
    uniqueConstraints = @UniqueConstraint(
        columnNames = {"user_id", "token_id",
            "contract_addr", "chain_id"}))
@Getter
@NoArgsConstructor(access = AccessLevel.PROTECTED)
public class NftHolding {
    @Id @GeneratedValue(strategy = SEQUENCE,
        generator = "nft_holding_seq")
    private Long id;
    private String userId; // 64자
    private Long tokenId;
    private String contractAddr; // 42자
    private Integer chainId;
    private Long amount;
    private Instant acquiredAt;
    private Instant releasedAt; // null = 현재 보유
    private String onChainTx; // TX 해시 66자
}
```

설계 결정 포인트

- 1 released_at = NULL → 현재 보유 중
released_at ≠ NULL → 소각·이전됨. DELETE 없음
- 2 Unique(userId, tokenId, contractAddr, chainId)
동일 토큰 중복 INSERT 방지 — 冪等性 확보
- 3 onChainTx 보관 → 감사 시 블록익스플로러 직접 검증 가능
- 4 역방향 동기화 절대 금지
오프체인 → 온체인 쓰기 없음. 온체인이 단일 진실

AuditLogEntry @Entity

audit_log 테이블 · append-only · prevChecksum Hash chain

```
@Entity
@Table(name = "audit_log")
@Getter @NoArgsConstructor(access = PROTECTED)
public class AuditLogEntry {
    @Id @GeneratedValue(strategy = SEQUENCE)
    private Long id;
    private Instant eventTime;
    private String actor; // 64자
    private String action; // 64자
    private String resourceType; // 32자
    private String resourceId; // 128자
    @JdbcTypeCode(SqlTypes.JSON)
    private String beforeState; // jsonb, nullable
    @JdbcTypeCode(SqlTypes.JSON)
    private String afterState; // jsonb, not null
    private String prevChecksum; // 이전 레코드 체크섬
    private String checksum; // SHA-256(64자)
}
```

Hash Chain 구조

첫 레코드: prevChecksum = "0000...0" (64자리 genesis)

이후 레코드: prevChecksum = 직전 레코드의 checksum

레코드 1: checksum = SHA-256(GENESIS + ...)

레코드 2: checksum = SHA-256(레코드1.checksum + ...)

레코드 3: checksum = SHA-256(레코드2.checksum + ...)

↓

레코드 N 번조 → 레코드 N+1의 prevChecksum 불일치 → 탐지

S26(Node.js)와 차이: Node.js AuditLogService는 단일 레코드 체크섬.

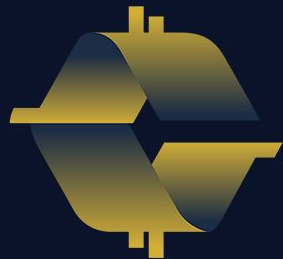
Java는 **chain** — prevChecksum으로 순서·연속성까지 보장

3 Service 구현

InternalLedgerService · AuditLogService · REQUIRES_NEW

@Transactional

SHA-256



COINCRAFT
ACADEMY

InternalLedgerService.recordNftHolding()

중복 방지 → JPA save → 감사 로그 자동 기록

internal-ledger/.../service/InternalLedgerService.java · **recordNftHolding(String userId, NftHoldingRequest request)**

① 중복 방지 (역등성)

existsBy... 쿼리로

Unique(userId + tokenId + contractAddr +
chainId) 확인

이미 존재하면 warn 로그 후 return — 예외 없
음

② JPA save()

NftHolding.of() 팩토리로 엔티티 생성

→ nftHoldingRepository.save()

→ 운영 DB INSERT

@Transactional — 실패 시 전체 롤백

③ 감사 로그 자동 기록

auditLogService.log() 호출

→ REQUIRES_NEW TX (별도 트랜잭션)

메인 TX 롤백과 무관하게 audit_log 보존

왜 REQUIRES_NEW가 필요한가 — NFT save 실패 → 메인 TX 롤백 → 그래도 "시도했다"는 감사 로그는 남아야 함

AuditLogService.log() — REQUIRES_NEW 트랜잭션

별도 트랜잭션 · PESSIMISTIC_WRITE · Hash chain 순서 보장

```
internal-ledger/.../service/AuditLogService.java · @Transactional(propagation = REQUIRES_NEW) log(...)
```

REQUIRES_NEW vs REQUIRED

Propagation	동작	감사 로그 결과
REQUIRED	메인 TX에 합류	메인 TX 롤백 → 감사 로그도 롤백 ✗
REQUIRES_NEW	별도 TX 생성	메인 TX 롤백 무관 → 감사 로그 보존 ✓

실행 순서

- ① Instant.now() 기록
- ② findLastForUpdate() — PESSIMISTIC_WRITE로 마지막 레코드 잠금
- ③ prevChecksum 추출 (없으면 GENESIS "0000...0")
- ④ computeChecksum() → INSERT

PESSIMISTIC_WRITE (SELECT FOR UPDATE)

동시에 두 스레드가 audit_log INSERT 시도 시
한 쪽이 잠금 획득 → 나머지 대기
→ 순차 처리 → prevChecksum 순서 보장

Hash Chain 무결성

레코드 N 변조 →
레코드 N+1의 prevChecksum 불일치 → 탐지
단일 레코드 수정만으로는 체인 속일 수 없음

computeChecksum() — SHA-256 구현

Java MessageDigest · HexFormat (Java 17+)

```
private String computeChecksum(  
    String prevChecksum,  
    String eventTime,  
    String actor,  
    String action,  
    String resourceId,  
    String afterState) {  
  
    String input =  
        prevChecksum + eventTime +  
        actor + action +  
        resourceId + afterState;  
  
    MessageDigest digest =  
        MessageDigest.getInstance("SHA-256");  
    byte[] hash = digest.digest(  
        input.getBytes(StandardCharsets.UTF_8));  
  
    return HexFormat.of().formatHex(hash);  
}
```

S26(Node.js)와 비교

항목	Node.js (S26)	Java (S26-2)
라이브러리	<code>crypto.createHash</code>	<code>MessageDigest</code>
입력값	단일 레코드 필드	prevChecksum + 필드
Chain	없음 (단독 체크섬)	있음 (hash chain)
잠금	PQueue 직렬화	PESSIMISTIC_WRITE

verifyIntegrity(auditLogId)

레코드 꺼내서 computeChecksum 재계산 → 저장된 checksum과 비교
불일치 = 변조 감지. 외부 감사 시 사용.

InternalGatewayClient — Node.js에서 Java를 부르는 방법

Node.js 18+ built-in fetch · X-Internal-Secret 헤더 · 내부망 전용

core-banking/src/adapters/InternalGatewayClient.ts · `recordNftHolding(userId, req)` · `recordAuditLog(req)`

X-Internal-Secret 헤더

하드코딩 금지

환경변수 `INTERNAL_GATEWAY_SECRET` 에서 주입

향후 mTLS로 전환 예정 (Phase 2)

Java DTO mirror 타입

`GatewayNftHoldingRequest`

`GatewayAuditLogRequest`

Java record와 필드·타입 1:1 대응

Java Instant → TS ISO-8601 string

내부망 전용

방화벽 레벨에서 외부 접근 차단

Node.js issuer-service만 호출 허용

Java Controller는 내부망 IP에서만 수신

void 엔드포인트

Java Controller

`ResponseEntity<Void>` 반환

content-type 확인 후 body 미파싱

200 OK = 성공

FINALIZED → 운영 DB 전체 호출 스택

on-chain 이벤트 확정부터 영구 원장 기록까지 단계별 흐름

- 1 **BlockchainEventListener** — Transfer(from=0x0) 이벤트 수신 → CONFIRMED → FINALIZED 전이
- 2 **LedgerService.finalize()** — mint_requests 상태 FINALIZED 업데이트 (PostgreSQL)
- 3 **KyoboCoreBankingAdapter.recordNftHolding()** — ICoreBankingAdapter 구현체 호출
- 4 **InternalGatewayClient.recordNftHolding()** — Circuit Breaker 통과 → `POST /api/internal/users/{userId}/nft-holdings`
- 5 **BlockchainGatewayController** — @Valid 검증 → `ledgerService.recordNftHolding()` 위임
- 6 **InternalLedgerService** — 중복 방지 → `nftHoldingRepository.save()` → 운영 DB INSERT
- 7 **AuditLogService.log()** — REQUIRES_NEW TX → prevChecksum 조회 → SHA-256 → 운영 DB INSERT

PostgreSQL(임시) → HTTP → 운영 DB(영구) 경계가 3→4 사이. 이 경계를 넘는 순간 가상자산이용자보호법 §15 보존 의무 테이블에 기록된다.

M4 S26-2 정리

internal-ledger Java 구현 핵심 4가지

① 역할 분리

Node.js = 블록체인 이벤트 처리(빠름)

Java = 금융 규제 영구 저장(안전)

API 계약(HTTP)으로 경계 명확히

② REQUIRES_NEW 감사 로그

메인 TX 롤백 ≠ 감사 로그 롤백

"시도했다"는 사실이 규제 증거

PESSIMISTIC_WRITE로 hash chain 순서 보장

③ 멍등성 (Idempotency)

```
existsByUserIdAndTokenId...
```

동일 TX 이벤트 재수신 → 중복 INSERT 없음

네트워크 재시도 안전

④ Circuit Breaker

Java 장애 → Node.js 요청 폭주 방지

5회 실패 → OPEN → 30초 쿨다운

상위 레이어 즉시 fallback 처리 가능

다음: S54 — ReconcileAdmin (Hourly·Daily·Manual) + Cron 스케줄링 + 운영 알림